Agenda

- Saída de dados: `print()`
- Entrada de dados: `input()`
- Expressões e Operadores Aritméticos
- Conversão de Tipos

A Função print()

Escrevendo na Tela: `print()`

- Para imprimir um texto na tela, utilizamos o comando `print()`.
- O texto pode ser um objeto do tipo **string**.

```
>>> print("De novo isso?")
De novo isso?
```

- No meio da **string** pode-se incluir caracteres de formatação especiais.
- O símbolo especial `\n` é responsável por pular uma linha na saída.

```
>>> print("De novo isso? \n Olá Pessoal!")
De novo isso?
Olá Pessoal!
```

Escrevendo o Conteúdo de um Objeto na Tela

- Podemos imprimir, além do conteúdo de objetos do tipo string, o conteúdo de outros tipos de objetos utilizando o comando **`print()`**.
- Também podemos separar múltiplos objetos a serem impressos com uma vírgula.

```
>>> print("O segundo parâmetro é um objeto do tipo <int> com o  valor ", 10)
O segundo parâmetro é um objeto do tipo <int> com o  valor  10
```

Escrevendo o Conteúdo de um Objeto na Tela

- Quando o parâmetro do comando print for uma variável, o conteúdo do objeto que é apontado (referenciado) pela variável será impresso.

```
>>> msg = "A variável a aponta para um objeto do tipo int com valor = "
>>> a = 10
>>> print(msg,a)
A variável a aponta para um objeto do tipo int com valor =  10
```

Escrevendo o Conteúdo de um Objeto na Tela

- Quando o parâmetro do comando print for uma variável, o conteúdo do objeto que é apontado (referenciado) pela variável será impresso.

```
>>> print("O tipo do objeto é ", type(a), " e seu valor é ", a)
```

O que será escrito na tela?

Escrevendo o Conteúdo de um Objeto na Tela

- Quando o parâmetro do comando print for uma variável, o conteúdo do objeto que é apontado (referenciado) pela variável será impresso.

```
>>> print("O tipo do objeto é ", type(a), " e seu valor é ", a)
O tipo do objeto é  <class 'int'>  e seu valor é  10
```

Escrevendo o Conteúdo de um Objeto na Tela

- A impressão com múltiplos argumentos inclui um **espaço extra** entre cada argumento.

```
>>> a = 10
>>> b = 3.14
>>> print("a contém o valor", a, "e b contém o valor", b)
a contém o valor 10 e b contém o valor 3.14
>>> print("a contém o valor ", a, " e b contém o valor ", b)
a contém o valor  10  e b contém o valor  3.14
```

Escrevendo o Conteúdo de um Objeto na Tela

- Podemos usar o comando **`str()`** para converter todos os objetos em objetos do tipo string e usar o **operador +** para concatenar strings de forma a imprimir sem estes espaços:

```
>>> a = 10
>>> b = 3.14
>>> print("a contém o valor" + str(a) + "e b contém o valor" + str(b))
a contém o valor10e b contém o valor3.14
>>> print("a contém o valor " + str(a) + " e b contém o valor " + str(b))
a contém o valor 10 e b contém o valor 3.14
```

Formatação de strings (Ex. com ponto flutuante)

- Podemos criar strings com formato específico usando caracteres especiais e o operador **%** .
- No exemplo abaixo, usamos os caracteres **%.Nf** dentro da string para informar que esta parte da string deve ser substituída por um valor de ponto flutuante e que apenas as N primeiras casas decimais devem ser impressas. No exemplo abaixo, N = 2:

```
>>> pi = 3.1415
>>> str = "Valor simplificado de pi: %.2f" % (pi)
>>> print(str)
Valor simplificado de pi: 3.14
```

Formatação de strings (Ex. com ponto flutuante)

- Você pode inclusive usar múltiplos argumentos para formatar a string:

```
>>> pi = 3.1415
>>> str = "O valor de pi e %.2f e seu quadrado e %.3f" % (pi, pi*pi)
>>> print(str)
O valor de pi e 3.14 e seu quadrado e 9.869
```

Formatação de strings (Ex. com ponto flutuante)

- Você pode inclusive usar múltiplos argumentos para formatar a string:

```
>>> pi = 3.1415
>>> str = "O valor de pi e %.2f e seu quadrado e %.3f" % (pi, pi*pi)
>>> print(str)
O valor de pi e 3.14 e seu quadrado e 9.869
```

- Você também pode produzir os "pedaços" da string de forma individual e concatená-los com o operador +:

```
>>> pi = 3.1415
>>> str = "O valor de pi e %.2f" % pi + " e seu quadrado e %.3f" % (pi*pi)
>>> print(str)
O valor de pi e 3.14 e seu quadrado e 9.869
```

Imprimindo sem Pula Linha

- A função **`print()`** sempre pula uma linha ao final da impressão.
- Se você não quiser que pule uma linha, inclua o argumento **end=' '** no **`print()`**.

```
>>> print("3, ", end="")
>>> print("4, ", end="")
>>> print("5 ", end="")
3, 4, 5
```

A Função input()

A Função `input()`

- Realiza a leitura de dados a partir do teclado.
- Aguarda que o usuário digite um valor e atribui o valor digitado a um objeto na memória.
- **Todos** os dados lidos são do tipo **string**.

```
>>> print("Digite um número: ")
>>> numero = input()
>>> print("O número digitado é: " + numero)
```

A Função `input()`

- Podemos converter uma string lida do teclado em um número inteiro usando a função **int()**.

```
>>> print("Digite um número: ")
>>> numero = int(input())
>>> numero = numero * 10
>>> print("O número digitado vezes 10 é: ", numero)
```

A Função `input()`

- Podemos fazer o mesmo para números ponto flutuante usando a função **float()**.

```
>>> print("Digite um número: ")
>>> numero = float(input())
>>> numero = numero * 10
>>> print("O número digitado vezes 10 é %.2f " %numero)
```

A Função `input()`

- Nos dois exemplos anteriores é esperado que o usuário digite um número.
- Se o usuário digitar um texto não númerico o programa encerrará com um erro de execução.

```
>>> print("Digite um número: ")
Digite um número:
>>> numero = float(input())
mc102
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: could not convert string to float: 'mc102'
```

A Função `input()`

- O programa abaixo lê dois números e imprime a soma destes.
- Perceba que podemos incluir um texto a ser impresso diretamente no comando `input()`.

```
>>> numero1 = float(input("Digite um número: "))
>>> numero2 = float(input("Digite um número: "))
>>> print("A soma dos números é: %.2f" %(numero1 + numero2))
```

Expressões

Expressões

- Já vimos que constantes e variáveis são expressões.
- Uma expressão também pode ser um conjunto de operações aritméticas, lógicas ou relacionais utilizadas para fazer "cálculos" sobre os valores dos objetos.
  - Exemplo: a + b

Expressões Aritméticas

- Os operadores aritméticos são: `+`, `-`, `*`, `/`, `//`, `%`, `**`

- **Adição**: expressão + expressão
- **Subtração**: expressão – expressão
- **Multiplicação**: expressão * expressão

```
>>> 30 + 5
35
>>> 30 - 5
25
>> 30 * 5
150
```

Expressões Aritméticas

- **Divisão**: expressão / expressão
  - O resultado é sempre um número ponto flutuante.
- **Divisão**: expressão // expressão
  - Se os operandos forem inteiros, a divisão é inteira. Se um deles for ponto flutuante faz uma divisão truncada.

```
>>> 5 / 2
2.5
>>> 5 // 2
2
>>> 5 // 2.0
2.0
```

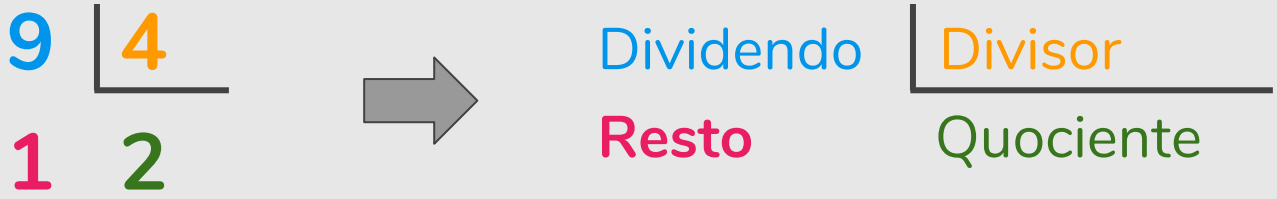
Expressões Aritméticas

- **Exponenciação (potenciação)**: expressão ** expressão
  - Calcula o valor da expressão à esquerda elevado ao valor da expressão à direita.
  - $a^n = a \times a \times a \times \ldots \times a$ (n vezes)

```
>>> 2 ** 4
16
>>> 2.2 ** 4
23.425600000000006
```

Expressões Aritméticas

- **Resto da Divisão**: expressão % expressão
  - Calcula o resto da divisão inteira de duas expressões.



Dividendo = Divisor * Quociente + **Resto**

Expressões Aritméticas

- **Resto da Divisão**: expressão % expressão
  - Calcula o resto da divisão inteira de duas expressões.

```
>>> 5 % 2
1
>>> 9 % 7
2
>>> 2 % 5
2
>>> 4 % 2
0
```

Expressões Aritméticas

- Exemplo: Converter segundos em horas, minutos e segundos.
  - 87426 segundos = horas, minutos e segundos?

```
>>> segundos_str = input("Por favor, digite o número de segundos que deseja converter: ")
>>> total_segundos = int(segundos_str)
>>> horas = total_segundos // 3600
>>> segundos_restantes = total_segundos % 3600
>>> minutos =  segundos_restantes // 60
>>> segundos_restantes_final = segundos_restantes % 60
>>> print("Horas =", horas, "minutos =", minutos, "segundos =" , segundos_restantes_final)
```

Expressões

- As expressões aritméticas (e todas as expressões) operam sobre outras expressões.
- É possível compor expressões complexas como por exemplo
  a = b * ((2 / c) + (9 % d * 8))

```
>>> 5 + 10 % 3
6
>>> 5 * 10 % 3
2
```

Precedência

- Precedência é a **ordem** na qual os operadores serão avaliados quando o programa for executado.
- Em Python, os operadores são avaliados na seguinte ordem:
  1. **
  2. *, /, //, na ordem em aparecerem na expressão
  3. %
  4. +, -, na ordem em aparecerem na expressão
- Exemplo: 8 + 10 * 6 é igual a 68

Precedência

- Operadores com a mesma precedência são executados da esquerda para a direita.
  - 6-3+2 é igual a ?
    - Esquerda para direita: 6-3 é igual a **3**, **3**+2 é igual a 5
    - ~~Direita para esquerda: 3+2 é igual a **5**, 6-**5** é igual a 1~~
- **Atenção**: Uma exceção é o operador exponenciação **.
  - 2 ** 3 ** 2 é igual a ?
    - ~~Esquerda para esquerda: 2 ** 3 é igual a **8**, **8** ** 2 é igual a 64~~
    - Direita para esquerda: 3 ** 2 é igual a **9**, 2 ** **9** é igual a 512

Alterando a Precedência

- **(expressão)** também é uma expressão, que calcula o resultado da expressão dentro dos parênteses, para só então calcular o resultado das outras expressões.
  - 5 + 10 % 3 é igual a 6
  - (5 + 10) % 3 é igual a 0

- Você pode usar quantos parênteses desejar dentro de uma expressão.
- Use sempre parênteses em expressões para deixar claro em qual ordem a expressão é avaliada!

Conversão de Tipos

Conversão de Tipos

- Já vimos o uso das funções **int()**, **float()** e **str()** que servem para converter dados de um tipo no outro especificado pela função.
- A conversão só ocorre se o dado estiver bem formado. Por exemplo **int("aaa")** resulta em um erro.
- Ao convertermos um número float para int ocorre um truncamento, ou seja, toda parte fracionária é desconsiderada.

Conversão de Tipos

```
>>> a = "ola"
>>> int(a)
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: ’ola’
>>> int(2.99)
2
>>> int(-2.99)
-2
>>> float("3.1415")
3.1415
```

Exercícios

- https://panda.ime.usp.br/pensepy/static/pensepy/02-Conceitos/conceitos.html#exercicios